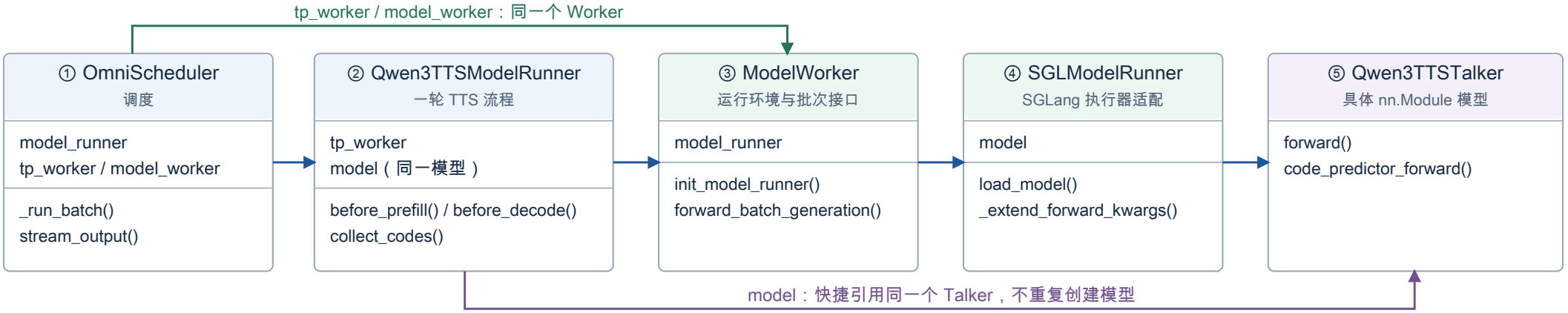
推理组件：对象持有与类继承

六个名字，七个实际类：本项目与 SGLang 各有一个 ModelRunner。

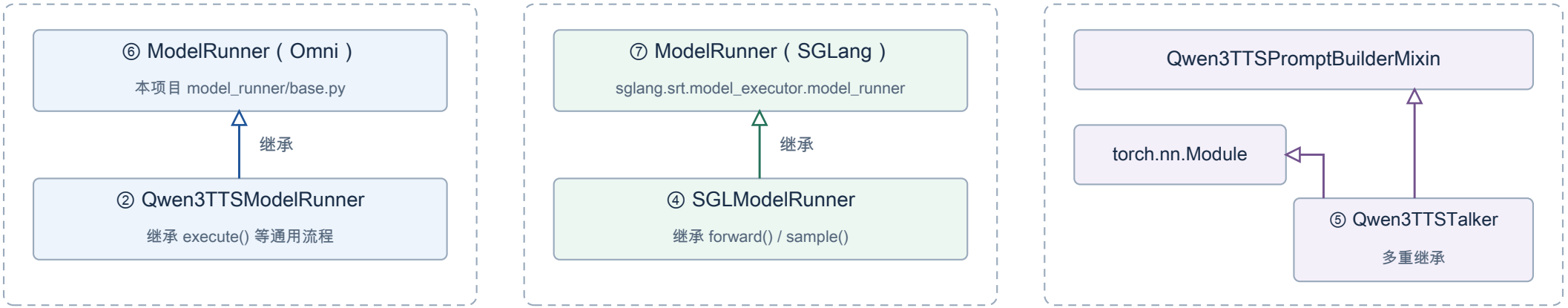
01 对象持有关系

实线箭头：属性引用；多条引用可指向同一实例



02 类继承关系

空心三角指向父类；省略 Generic 与隐式 object



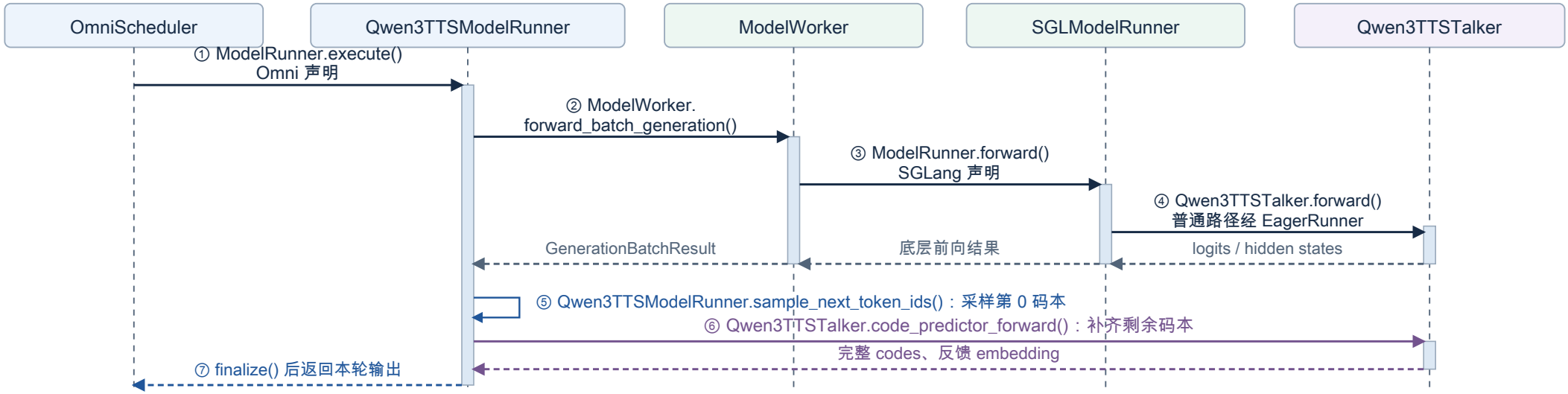
读图。 实线箭头表示属性持有的可导航关联；空心三角指向父类。这里的“组合”按属性引用理解，Worker 和模型实例可以被多个对象引用。省略 Generic[...] 与隐式 object 基类。

同一实例。 OmniScheduler.tp_worker、OmniScheduler.model_worker 与 Qwen3TTSModelRunner.tp_worker 指向同一个 Worker；外层 Runner 的 model 与 tp_worker.model_runner.model 指向同一个 Talker。多条引用不代表加载了多份模型。

继承。 Qwen3TTSModelRunner 继承本项目 ModelRunner；SGLModelRunner 继承 SGLang ModelRunner；Qwen3TTSTalker 同时继承 Qwen3TTSPromptBuilderMixin 与 torch.nn.Module。OmniScheduler 通过 __getattr__ 绑定并复用上游 Scheduler 的方法，没有继承它；ModelWorker 没有自定义父类。

一轮计算：调用顺序与各类职责

对象在启动时创建并复用；箭头上的方法名前缀使用实际声明该方法的类。



⑦ 的 finalize() 声明在本项目 ModelRunner；子类调用继承方法时，self 仍是子类实例。

类	在一次 TTS 中的职责	核心方法及声明位置
① OmniScheduler	请求入队、组批，反复推进生成；完成后将最终结果放入 outbox。	自己声明 _run_batch()、process_batch_result()、stream_output()。
② Qwen3TTSModelRunner	准备 TTS 输入，参与采样；调用 Code Predictor 补齐码本，保存 codes 和下一轮反馈。	自己声明 before_prefill()、before_decode()、collect_codes()；execute() 继承自⑥。
③ ModelWorker	启动时初始化设备、并行环境及底层 Runner；请求中执行批次，提供资源管理接口。	自己声明 init_model_runner()、forward_batch_generation()、get_memory_pool()。
④ SGLModelRunner	将本项目模型接入 SGLang：注册、加载模型，对接 Prefill 输入、CUDA Graph 与 KV 预算。	自己声明 load_model()、_extend_forward_kwargs()、init_cuda_graphs()；forward() 继承自⑦。
⑤ Qwen3TTSTalker	定义模型结构；主前向计算 logits、hidden states；Code Predictor 生成剩余码本。	自己声明 forward()、code_predictor_forward()；主前向内部运行 TextModel 与 DecoderLayer。
⑥ ModelRunner（Omni）	定义一轮生成的通用流程：构建输入、调用前后置钩子、前向、采样、整理输出。	本项目 model_runner/base.py：execute()、prepare_and_forward()、finalize()。
⑦ ModelRunner（SGLang）	提供通用前向执行、普通执行与图执行分流，以及 token 采样能力。	依赖包 sglang.srt.model_executor.model_runner：forward()、_forward_raw()、sample()。

轮次与输出。 一次 execute() 通常是一轮自回归步骤，可处理多条请求，不等于整段语音生成完成。Scheduler 反复推进到结束条件；完成请求由 stream_output() 输出最终结果，Stage 交给后续 vocoder 将 codes 转为波形。

CUDA Graph。 时序图展开普通主模型前向；命中图时，底层 Runner 执行已捕获的计算，不保证重新进入 Python forward()。Code Predictor 也有自己的图路径。